VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MATHEMATICAL MODELING REPORT

Symbolic and Algebraic Reasoning in Petri Nets

CLASS CC02 - GROUP 35

No.	STUDENT NAME	ID
1	Nguyễn Trung Vĩnh	2453438
2	Phạm Vũ Khôi Nguyên	2412372
3	Lê Minh Nhật	2452891

Ho Chi Minh City, November 2025

Contents

1 Introduction **2**

2 Theoretical Background **2**
2.1 1-safe Petri Nets and Reachability 2
2.2 Explicit Reachability Computation 3
2.3 BDDs and Symbolic State-Space Exploration 4
2.4 ILP-based Deadlocks and BDD-based Optimization 4

3 Implementation Design and Data Structures **5**
3.1 PNML Parsing 5
3.2 Petri Net Data Structure 6
3.3 Explicit Reachability (Task 2) 6
3.4 Symbolic Reachability with BDDs (Task 3) 7
3.5 Deadlock Detection (Task 4) 8
3.5.1 ILP-based search for dead markings. 8
3.5.2 BDD-based reachability check. 8
3.5.3 Fallback symbolic deadlock detection. 9
3.6 Optimization over Reachable Markings (Task 5) 9
3.7 Main Program and Example Models 10

4 Experimental Results and Discussion **10**
4.1 Experimental Setup 10
4.2 Deadlock Analysis 11
4.3 Optimization Results 11
4.4 Explicit vs. Symbolic Reachability 12

5 Challenges and Possible Improvements **12**
5.1 PNML parsing and namespaces. 12
5.2 1-safety and matrix dimensions. 13
5.3 BDD variable naming and library compatibility. 13
5.4 ILP formulations. 13
Potential improvements 13

6 Conclusion **14**

References **15**

1. Introduction

Petri Nets are a classical formalism for modeling concurrent, distributed, and event-driven systems. They provide a compact graphical language where *places* represent conditions, *transitions* represent events, and *tokens* represent the dynamic state of the system. The evolution of the system is captured by *firing* transitions, which consume and produce tokens according to a fixed flow relation.

Among various subclasses, *1-safe* Petri nets are particularly attractive for state-space exploration: each place contains at most one token, so every marking can be represented by a Boolean vector. Important verification questions such as reachability, deadlock-freedom or optimization over markings can then be formulated over this Boolean state space.

However, explicit enumeration of all reachable states quickly becomes intractable as the number of places and transitions increases. To address this difficulty, symbolic methods based on Binary Decision Diagrams (BDDs) are commonly used to compactly represent large sets of states, and Integer Linear Programming (ILP) formulations are used to reason about reachability and optimization properties.

This report documents our implementation of a small analysis tool satisfying these requirements. The main contributions of the tool are:

- A PNML parser that constructs an internal 1-safe Petri net model;
- an explicit breadth-first search (BFS) algorithm to enumerate reachable markings;
- A BDD-based symbolic reachability procedure;
- A deadlock-analysis module operating directly on the symbolic representation;
- A simple optimization module that maximizes a linear objective function over reachable markings.

All components are implemented in Python and integrated in a single driver program that can be applied to PNML models of small or medium size.

2. Theoretical Background

2.1. 1-safe Petri Nets and Reachability

A (place/transition) Petri net is a tuple

$$N = (P, T, F, M_0),$$

where P is a finite set of places, T is a finite set of transitions, $F \subseteq (P \times T) \cup (T \times P)$ is a set of directed arcs, and $M_0 : P \to \mathbb{N}$ is the initial marking assigning a number of tokens to each place. For a place $p \in P$, $M(p)$ denotes the number of tokens at p in marking M.

For a transition $t \in T$, we denote its preset and postset by

$$^\bullet t = \{\, p \in P \mid (p,t) \in F \,\}, \qquad t^\bullet = \{\, p \in P \mid (t,p) \in F \,\}.$$

A transition $t \in T$ is *enabled* at a marking M if

$$\forall p \in P :\ M(p) \geq W^-(p,t).$$

When an enabled transition t fires once, a new marking M' is produced by

$$\forall p \in P :\quad M'(p) = M(p) - W^-(p,t) + W^+(p,t).$$

The *reachability relation* is the reflexive–transitive closure of this one-step firing relation. A marking M is called *reachable* if it can be obtained from M_0 by firing a finite sequence of transitions. The set of all reachable markings is denoted by $Reach(M_0)$.

A Petri net is *1-safe* if every reachable marking satisfies $M(p) \in \{0,1\}$ for all places $p \in P$. In this case, a marking can be identified with a Boolean vector of length $|P|$, which is instrumental for both explicit search and BDD-based representation.

2.2. Explicit Reachability Computation

The most direct way to construct $Reach(M_0)$ is to explore the state space explicitly. Starting from the initial marking, one repeatedly fires all enabled transitions and stores each previously unseen marking. A standard breadth-first search (BFS) algorithm ensures that every reachable marking is eventually visited and that the length of the firing sequence to each marking corresponds to its BFS depth.

In our implementation, markings are represented as integer vectors of length $|P|$, and the structural information of the net is encoded in two integer matrices:

- The *pre-matrix* $I \in \mathbb{N}^{|T|\times|P|}$, where $I_{t,p}$ is the number of tokens consumed from place p when firing transition t;
- The *post-matrix* $O \in \mathbb{N}^{|T|\times|P|}$, where $O_{t,p}$ is the number of tokens produced on place p by transition t.

The enabling condition becomes

$$M \geq I_t \quad \text{(component-wise)},$$

and the firing rule can be written compactly as

$$M' = M - I_t + O_t.$$

For a 1-safe net, we additionally require $M'(p) \leq 1$ for all p, and discard any successor violating this property.

2.3. BDDs and Symbolic State-Space Exploration

A Binary Decision Diagram (BDD) is a canonical representation of a Boolean function $f(x_1, \ldots, x_n)$ as a directed acyclic graph with non-terminal nodes labelled by variables and terminal nodes labelled by 0 or 1. Reduced, ordered BDDs offer a compact representation for many practically relevant Boolean functions and support efficient logical operations such as conjunction, disjunction, negation, and existential quantification.

In a 1-safe Petri net, each place $p_i \in P$ naturally corresponds to a Boolean variable x_i, where $x_i = 1$ means that p_i carries a token. Therefore, each marking M corresponds to a unique valuation of the vector $x = (x_1, \ldots, x_{|P|})$. A set of reachable markings $S \subseteq \{0,1\}^{|P|}$ can be represented by the Boolean function

$$f_S(x) = 1 \iff x \in S$$

and implemented as a BDD. The set of all reachable markings can thus be constructed symbolically by iterating the following *image computation* step:

1. Take the current set R of already known reachable states;
2. Apply the transition relation to obtain all one-step successors $Post(R)$;
3. Update $R \leftarrow R \cup Post(R)$ until a fixed point is reached.

Crucially, all operations are performed on BDDs instead of explicitly enumerated state sets, which may yield significant memory savings when the state space has regular structure.

2.4. ILP-based Deadlocks and BDD-based Optimization

A marking M is called a *dead marking* if no transition is enabled at M. A *deadlock* is a dead marking that is reachable from M_0. In our setting, a deadlock therefore satisfies two conditions:

- Reachability: $M \in (M_0)$, and
- Disabling: for all $t \in T$, M does not enable t.

The assignment suggests using Integer Linear Programming (ILP) together with symbolic information to detect deadlocks and to optimize linear objective functions over reachable

markings. Conceptually, one introduces binary decision variables corresponding to places (and possibly to transition firings), and imposes linear constraints encoding both the reachability region and the disabling condition. Optimizing a linear objective function $c^\top M$ over this polytope amounts to finding a reachable marking that maximizes the weighted sum of tokens with weight vector $c \in \mathbb{Z}^{|P|}$.

In our implementation, we combine ILP and BDD. For the deadlock detection task, we explicitly build an ILP model using the PuLP library. The model contains a binary variable $M[p]$ for each place and a non-negative integer variable $x[t]$ for each transition. The state equation

$$M = M_0 + Cx$$

is enforced component-wise, where C is the incidence matrix $C[t, p] = Post[t, p] - Pre[t, p]$. In addition, for every transition t with at least one input place, we impose

$$\sum_{p \in I(t)} M[p] \leq |I(t)| - 1,$$

which ensures that at least one input place is empty, and hence t is disabled in M. Solving this ILP with PuLP either yields a candidate dead marking M or proves that no such marking satisfies the constraints.

The candidate marking returned by the ILP is then checked against the BDD $Reach(M_0)$ obtained in Task 3: if the conjunction of the BDD encoding M with $Reach(M_0)$ is satisfiable, then M is a reachable dead marking (a deadlock). If the ILP solution is unreachable, we fall back to a purely BDD-based deadlock search over $Reach(M_0)$.

For the optimization task, instead of building a separate ILP, we directly evaluate the linear objective function $c^\top M$ over the satisfying assignments of the reachable-set BDD. This BDD-based enumeration is sufficient for the small and medium-size models considered in this assignment and keeps the implementation lightweight.

3. Implementation Design and Data Structures

3.1. PNML Parsing

The class `PNMLReader` encapsulates the logic of reading a PNML file and constructing a `SafePetriNetSystem`. It uses the standard `xml.etree.ElementTree` library to parse XML and carefully handles namespaces, which are common in PNML files.

The reader proceeds in three steps:

1. **Places.** It scans all `<place>` elements, extracting their identifier, display name (if present), and initial marking. These are stored in an ordered list so that the index

of a place is stable throughout the program.

2. **Transitions.** It scans all `<transition>` elements to obtain transition identifiers and display names.
3. **Arcs.** It scans all `<arc>` elements and inspects the `source` and `target` attributes. If the arc goes from a place to a transition, the corresponding entry in the pre-matrix is incremented; if it goes from a transition to a place, the post-matrix is updated. Optional inscription text is interpreted as an integer weight.

3.2. Petri Net Data Structure

The core model is the class `SafePetriNetSystem` with the following attributes:

- `places`: list of place identifiers;
- `transitions`: list of transition identifiers;
- `pre_matrix`, `post_matrix`: integer matrices of size $|T| \times |P|$;
- `initial_marking`: integer vector M_0;
- `place_labels` and `transition_labels` for readable output.

The method `get_enabled_transitions` implements the enabling condition using NumPy vector operations. For a current marking M, each row I_t of the pre-matrix is compared component-wise to decide whether $M \geq I_t$. The method `fire_transition` implements the update rule $M' = M - I_t + O_t$, which is exactly the vectorised version of the Petri net semantics.

3.3. Explicit Reachability (Task 2)

The function `run_bfs_explicit` performs a breadth-first search over the state space. It stores visited markings as immutable tuples in a Python `set` and uses a standard queue for BFS order.

The algorithm is:

1. Initialise the visited set with M_0 and push M_0 into the queue.
2. While the queue is not empty, pop one marking M.
3. For each transition t:
   (a) check if t is enabled at M using the pre-matrix;
   (b) compute $M' = M - I_t + O_t$;
   (c) if any component of M' is greater than 1, discard M' to preserve 1-safety;

(d) if M' has not been visited before, add it to the visited set and to the queue.

At the end, the function returns the set of all reachable markings and prints summary statistics such as the number of steps and the total number of states.

3.4. Symbolic Reachability with BDDs (Task 3)

The function `run_bdd_reachability` implements the symbolic algorithm using PyEDA. It first creates two Boolean BDD variables for each place: x_i for the current marking and y_i for the next marking. Since PNML identifiers can contain characters that are illegal in Python identifiers, variable names are passed through `sanitize_variable_name` before creating the BDD variables.

The initial marking M_0 is encoded as a conjunction over the x-variables: x_i if $M_0(p_i) = 1$ and $\neg x_i$ otherwise. This results in the initial BDD R representing the singleton set $\{M_0\}$.

For each transition t, two components are constructed:

- an *enable condition* over the current state variables x, which requires all input places to contain a token and all pure output places (those in $t^\bullet \setminus {}^\bullet t$) to be empty;
- a *next-state formula* relating x and y: an output place gets a token (y_i), an input-only place loses its token ($\neg y_i$), and all other places keep their token ($x_i \Leftrightarrow y_i$).

The conjunction of these two parts yields a BDD TR_t characterising all legal state transitions of t. The global transition relation is then

$$TR = \bigvee_{t \in T} TR_t.$$

Symbolic reachability is computed by iterating:

1. compute the relational product $R(x) \wedge TR(x, y)$;
2. existentially quantify over the current variables x (PyEDA `smoothing`) to obtain a BDD over the y-variables;
3. rename each y_i back to x_i;
4. union with the previous set R;
5. stop when a fixed point is reached.

The final BDD still represents the set $Reach(M_0)$, and PyEDA's `satisfy_count` method is used to count the number of reachable markings.

3.5. Deadlock Detection (Task 4)

The final implementation of Task 4 combines an ILP formulation with the BDD representation obtained in Task 3. Conceptually, a dead marking is a marking M in which no transition is enabled, and a deadlock is a dead marking that is reachable from the initial marking M_0.

3.5.1. ILP-based search for dead markings.

Given a 1-safe Petri net with incidence matrix C and initial marking M_0, we introduce:

- a binary variable $M[p] \in \{0, 1\}$ for each place $p \in P$,
- a non-negative integer variable $x[t] \in \mathbb{Z}_{\geq 0}$ for each transition $t \in T$.

The marking variables must satisfy the state equation

$$M[p] = M_0[p] + \sum_{t \in T} C[t, p] \cdot x[t] \quad \text{for all } p \in P,$$

which is a standard necessary condition for reachability in bounded Petri nets. To encode the fact that no transition is enabled, for every transition t with a non-empty input set $I(t)$ we require

$$\sum_{p \in I(t)} M[p] \leq |I(t)| - 1.$$

In a 1-safe net this guarantees that at least one input place of t is empty, hence t is disabled at M. The resulting mixed-integer linear system is solved using the PuLP library with the CBC backend. Any feasible solution corresponds to a candidate dead marking M.

3.5.2. BDD-based reachability check.

The ILP formulation uses only the state equation and disabling constraints, so it is an over-approximation of the reachable set. To ensure that the candidate marking is actually reachable, we reconstruct the BDD variables for the current marking and build a Boolean formula encoding the assignment M:

$$\varphi_M = \bigwedge_{p \in P} \begin{cases} x_p & \text{if } M[p] = 1, \\ \neg x_p & \text{if } M[p] = 0. \end{cases}$$

We then check whether $\varphi_M \wedge \mathit{Reach}(M_0)$ is satisfiable. If the conjunction is non-zero in PyEDA, the marking M is reachable and thus a true deadlock reachable from M_0.

3.5.3. Fallback symbolic deadlock detection.

If the ILP is infeasible, we conclude that no dead marking satisfies the state equation and the disabling constraints, and therefore the net is deadlock-free. If the ILP returns a marking that is not contained in $Reach(M_0)$, we fall back to a purely BDD-based procedure.

In the BDD phase, we reconstruct the current-state variables x_p for all places p and, for each transition t, build a Boolean condition $enabled_t(x)$ characterising when t is enabled. The condition uses the same logic as in the symbolic transition relation of Task 3: all input places of t must be marked, and all pure output places of t must be empty. We then define

$$AnyEnabled(x) = \bigvee_{t \in T} enabled_t(x), \qquad Dead(x) = \neg AnyEnabled(x),$$

and obtain the set of dead markings by intersecting with the reachable-set BDD:

$$DeadReach(x) = Reach(M_0)(x) \wedge Dead(x).$$

If *DeadReach* is non-zero, we extract one satisfying assignment as a concrete deadlock marking; otherwise, the system is reported as deadlock-free.

This two-level design allows the ILP solver to quickly rule out large portions of the search space, while the BDD guarantees correctness by restricting attention to truly reachable markings.

3.6. Optimization over Reachable Markings (Task 5)

The final module `find_optimal_marking` implements optimization of a linear objective function $f(M) = c^\top M$ over the reachable-set BDD. It takes as input the Petri net system, the reachable-set BDD, and a vector of integer weights c (stored as a NumPy array).

The procedure iterates over all satisfying assignments of the BDD using PyEDA's `satisfy_all` generator. Each assignment is in general a *cube*, i.e. a partial valuation where some variables may be "don't-care". For each variable x_i:

- if the cube assigns a fixed value, that value is taken as the bit in the current marking;
- if the variable is free, a greedy choice is made: $x_i = 1$ if the weight c_i is positive, otherwise $x_i = 0$.

This produces a concrete 0–1 mark for each cube, and the corresponding objective value $c^\top M$ is accumulated. The best value and marking are tracked and reported at the end. For the simple models considered in this assignment, this exhaustive approach is sufficient and avoids the complexity of integrating a full ILP solver.

3.7. Main Program and Example Models

The main entry point of the implementation is the script `main.py`. It provides a simple command-line interface that accepts the path of a PNML file:

```
python main.py --input tests/test_small.pnml
```

By default, if no argument is given, the program loads `tests/test_small.pnml`. The PNML file is parsed by the `PNMLReader` class (Task 1) into an instance of `SafePetriNetSystem`. The main program then executes, in order:

1. Task 2: explicit reachability by calling `run_bfs_explicit`, which prints the number of reachable markings, the number of BFS steps, and the execution time.
2. Task 3: symbolic reachability by calling `run_bdd_reachability`, which returns the reachable-set BDD and its cardinality. The main program compares the number of states obtained from BFS and from the BDD.
3. Task 4: deadlock detection by calling `find_deadlock`, which internally uses the ILP formulation with PuLP and falls back to a BDD analysis if needed.
4. Task 5: optimization by calling `find_optimal_marking` with a weight vector c; in our experiments we use the simple choice $c = (1, \ldots, 1)$ so that the objective counts the total number of tokens.

All numerical results reported in Section 4 are obtained by running `main.py` on two PNML models, `test_small.pnml` and `test_medium.pnml`, provided together with the source code.

4. Experimental Results and Discussion

4.1. Experimental Setup

All experiments are performed on small PNML models that are 1-safe by construction. The implementation is written in Python and relies on the NumPy library for numerical operations and PyEDA for BDD manipulation. The default entry point `main.py` is used to run the full pipeline for each model.

For each experiment, we record:

- the number of places and transitions;
- the total number of reachable markings;
- whether a deadlock is found;
- the value of the optimization objective for a given weight vector.

4.2. Deadlock Analysis

By definition, a deadlock is a reachable marking where no transition is enabled. In the small resource-allocation model `test_small.pnml`, the places represent the local state of two processes (`P1_Ready`, `P1_Has_Res1`, `P2_Ready`, `P2_Has_Res2`) and two shared resources (`Res_1`, `Res_2`). The initial marking $M_0 = [1, 0, 1, 0, 1, 1]$ has both processes ready and both resources available, so at least one transition is enabled (for instance, `P1_Take_R1` and `P2_Take_R2` are both enabled).

Among the four reachable markings computed in Tasks 2 and 3, the marking $M^d = [0, 1, 0, 1, 0, 0]$ is of particular interest. Here, process 1 holds `Res_1` and process 2 holds `Res_2`, while both resource places `Res_1` and `Res_2` are empty. In this configuration:

- `P1_Take_R1` is disabled because `P1_Ready` is 0;
- `P1_Take_R2` is disabled because `Res_2` is 0;
- `P2_Take_R2` is disabled because `P2_Ready` is 0;
- `P2_Take_R1` is disabled because `Res_1` is 0.

Thus no transition is enabled at M^d, and M^d is a deadlock marking. Since M^d is also contained in the reachable-set BDD obtained in Task 3, the ILP + BDD procedure from Task 4 correctly reports a reachable deadlock and returns the vector $[0, 1, 0, 1, 0, 0]$ as shown in the program output. This behaviour is consistent with the intuitive understanding of the system: each process is waiting for a resource currently held by the other one, and the computation cannot make any further progress.

Table 1: Deadlock analysis using ILP + BDD.

Model	Deadlock?	Example deadlock marking	Runtime (s)
`test_small.pnml`	Yes	$[0, 1, 0, 1, 0, 0]$	0.0565

4.3. Optimization Results

For the model `test_small.pnml`, we choose a uniform weight vector $w = [1, 1, 1, 1, 1, 1]$ over the six places. With this choice, the objective function $w^\top M$ is simply the total number of tokens in the marking (i.e., the number of marked places in a 1-safe net). When both resources are held by the processes, the resource places become empty, and the total token count can decrease. Therefore, different reachable markings may yield different objective values.

The optimization module enumerates satisfying assignments (solution cubes) of the reachable-set BDD obtained in Task 3 and evaluates $w^\top M$ for each candidate. In this

experiment, it scans 4 solution cubes, corresponding to the 4 reachable markings. The maximum objective value is 4, attained at the marking

$$M^\star = [1, 0, 1, 0, 1, 1],$$

which coincides with the initial marking M_0 (both processes are ready and both resources are available). The measured execution time is 0.0001 seconds, as reported in Table 2.

Table 2: Optimization over reachable markings.

Model	Weight vector w	Max $w^\top M$	Optimal marking	Runtime (s)
`test_small.pnml`	$[1, 1, 1, 1, 1, 1]$	4	$[1, 0, 1, 0, 1, 1]$	0.0001

4.4. Explicit vs. Symbolic Reachability

On such small examples, symbolic computation does not provide a performance advantage and even incurs some overhead due to BDD manipulation. Nevertheless, the structure of the implementation follows standard symbolic model-checking techniques and can scale better than explicit BFS when the state space grows and the reachable set exhibits regularity.

Conceptually, explicit BFS stores one concrete marking per reachable state, whereas the BDD-based method stores a single logical function representing the entire reachable set. For highly concurrent models where many markings share similar patterns, the BDD can remain compact even when the number of states is exponential in the number of places.

Table 3: Explicit BFS vs. BDD-based symbolic reachability.

Model	$\lvert Reach(M_0)\rvert$ (BFS)	BFS time (s)	$\lvert Reach(M_0)\rvert$ (BDD)	BDD iterations
`test_small.pnml`	4	0.00015	4	3

5. Challenges and Possible Improvements

During the development of this assignment, several practical challenges were encountered.

5.1. PNML parsing and namespaces.

Real PNML files often use XML namespaces. Without explicit handling, standard XPath queries fail to locate elements. The `PNMLReader` solves this by dynamically extracting the namespace URI from the root tag and prefixing all subsequent queries with an appropriate namespace alias.

5.2. 1-safety and matrix dimensions.

Although the assignment assumes that the input models are already 1-safe, the BFS procedure still explicitly checks that no successor marking contains more than one token in any place. This defensive programming style protects against malformed input. In the BDD code, additional care is taken when iterating over the rows and columns of the pre- and post-matrices to avoid out-of-bounds access when matrices are incomplete.

5.3. BDD variable naming and library compatibility.

The PyEDA library expects variable names to follow identifier-like conventions. To avoid errors from PNML identifiers containing special characters, all names are passed through `sanitize_variable_name`, which replaces non-alphanumeric characters by underscores and prefixes names that start with a digit. The utility module also patches a small compatibility issue between PyEDA and newer versions of Python's standard library.

5.4. ILP formulations.

In our final implementation, we follow this recommendation for Task 4 by constructing an ILP model and solving it with the PuLP library beforenconsulting the BDD. For Task 5, we deliberately keep the implementation BDD-based: the linear objective function is evaluated directly on the reachable set represented as a BDD, using a greedy treatment of "don't care" variables.

While the BDD-based optimization is sufficient for the small and medium-size models considered in this assignment, large-scale case studies would benefit from a full ILP formulation of the optimization problem on top of the reachable polytope.

Potential improvements

- Extending the ILP approach already used in Task 4 to also cover the optimization problem in Task 5, and systematically comparing its performance with the current BDD-based enumeration;
- Exploiting dynamic variable reordering in the BDD library to further reduce BDD size on larger models;
- Extending the parser and analysis to support coloured or timed Petri nets;
- Adding a small command-line interface or graphical front-end to select PNML files, objective functions, and analysis options.

6. Conclusion

We implemented a complete analysis pipeline for 1-safe Petri nets, including PNML parsing, explicit reachability enumeration, symbolic reachability using BDDs, ILP-based deadlock detection, and optimization over reachable markings. On the example models used in our experiments (e.g., `test_small.pnml`), the symbolic approach matches the explicit exploration in terms of reachable markings, and the ILP procedure successfully identifies a reachable deadlock when it exists. Finally, the optimization task is solved by enumerating satisfying cubes of the reachable-set BDD, which is adequate for the small and medium models required by the assignment.

References

[1] W. Reisig, *Understanding Petri Nets: Modeling Techniques, Analysis Methods, Case Studies*. Berlin, Germany: Springer, 2013.

[2] Enric Pastor, Jordi Cortadella, and Oriol Roig, *Symbolic Analysis of Bounded Petri Nets*, Access link at:
https://upcommons.upc.edu/server/api/core/bitstreams/d667887a-ec02-420c-9021-1b2155f98015/content